

Entendendo a ideia do React e criando uma aplicação que consulta uma API

Prof. Lucas Lima

Objetivo da aula

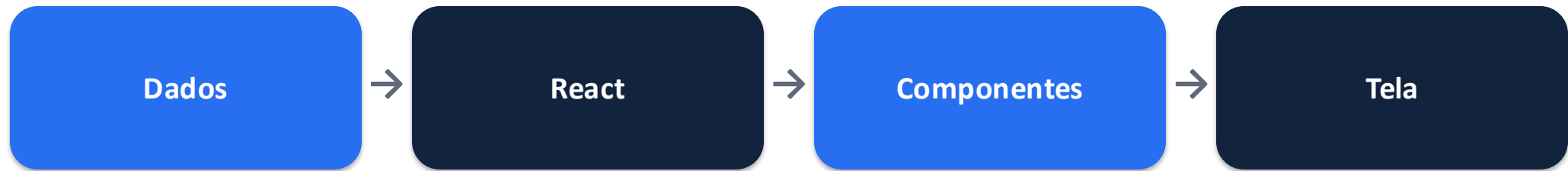
- Entender o que é React sem entrar em conceitos avançados
- Entender o que são componentes e como uma tela é construída
- Criar um projeto React usando Vite
- Conhecer useState e useEffect de forma simples
- Consultar uma API e mostrar os dados na tela

O que é React?

- React é uma biblioteca JavaScript usada para construir interfaces
- A aplicação é dividida em pequenos pedaços chamados componentes
- Quando os dados mudam, o React atualiza a interface
- A ideia é construir a tela a partir dos dados da aplicação

A ideia principal do React

Podemos pensar no React como a ligação entre os dados e aquilo que o usuário vê.



Antes do React

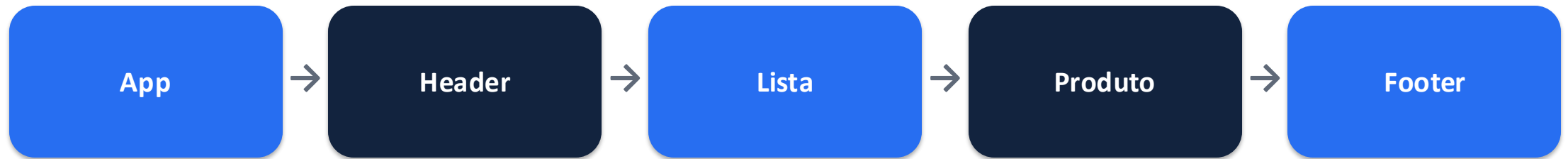
- HTML cria a estrutura da página
- CSS define a aparência
- JavaScript adiciona comportamento
- Com React continuamos usando JavaScript, mas organizamos a interface em componentes

O que é um componente?

- Um componente representa uma parte da interface
- Pode ser um cabeçalho, botão, produto, formulário ou página inteira
- Componentes podem ser reutilizados
- Em React, um componente pode ser criado como uma função JavaScript

Uma aplicação dividida em componentes

A aplicação maior é formada por componentes menores.



Nosso primeiro componente

Uma função que retorna aquilo que queremos mostrar na interface.

```
function Header() {  
  return <h1>Minha Loja</h1>;  
}  
  
export default Header;
```

O que é JSX?

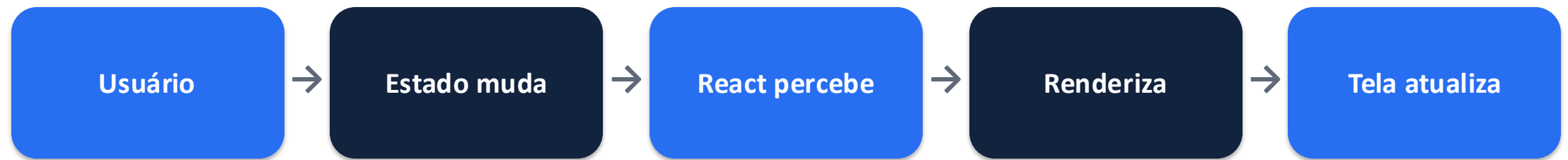
- JSX é a sintaxe usada para escrever a interface dentro do JavaScript
- Ele lembra HTML, por isso é mais fácil visualizar a tela
- Podemos colocar valores JavaScript usando chaves { }
- Não precisamos aprofundar JSX agora: vamos aprender usando

Exemplo simples de JSX

```
function App() {  
  const nome = "Lucas";  
  
  return (  
    <h1>Olá, {nome}</h1>  
  );  
}
```

Quando um dado muda

Esse fluxo é uma das ideias mais importantes para entender React.



Agora vamos criar o projeto

- Vamos usar Vite para criar a estrutura inicial
- Vite prepara o projeto React para nós
- Precisamos ter Node.js instalado
- Vamos desenvolver acompanhando o resultado no navegador

Criando o projeto com Vite

Execute os comandos no terminal.

```
npm create vite@latest minha-loja
```

```
# Escolha:
```

```
# React
```

```
# JavaScript
```

```
cd minha-loja
```

```
npm install
```

```
npm run dev
```

Arquivos importantes para começar

- `src/main.jsx` → inicia a aplicação React
- `src/App.jsx` → nosso componente principal
- `src/index.css` → estilos da aplicação
- `package.json` → informações, scripts e dependências do projeto

Simplificando o App.jsx

```
function App() {  
  return (  
    <div>  
      <h1>Minha Loja</h1>  
      <p>Produtos disponíveis</p>  
    </div>  
  );  
}  
  
export default App;
```

Criando um componente Produto

- Vamos separar um produto em seu próprio componente
- Isso deixa o App mais organizado
- Depois poderemos repetir o mesmo componente para vários produtos

Produto.jsx

```
function Produto() {  
  return (  
    <div>  
      <h2>Notebook</h2>  
      <p>R$ 3.500</p>  
    </div>  
  );  
}  
  
export default Produto;
```

Usando o componente no App

```
import Produto from "../components/Produto";

function App() {
  return (
    <div>
      <h1>Minha Loja</h1>
      <Produto />
    </div>
  );
}

export default App;
```

O que é estado?

- Estado é uma informação que pode mudar durante o uso da aplicação
- Exemplos: contador, nome digitado, lista de produtos
- Quando o estado muda, o React pode atualizar a tela
- Para começar, vamos conhecer o hook useState

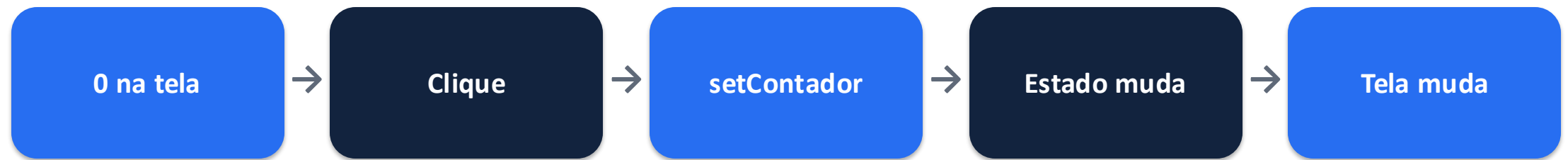
Primeiro exemplo com useState

```
import { useState } from "react";

function App() {
  const [contador, setContador] = useState(0);

  return (
    <button onClick={() => setContador(contador + 1)}>
      Cliques: {contador}
    </button>
  );
}
```

O que aconteceu no contador?

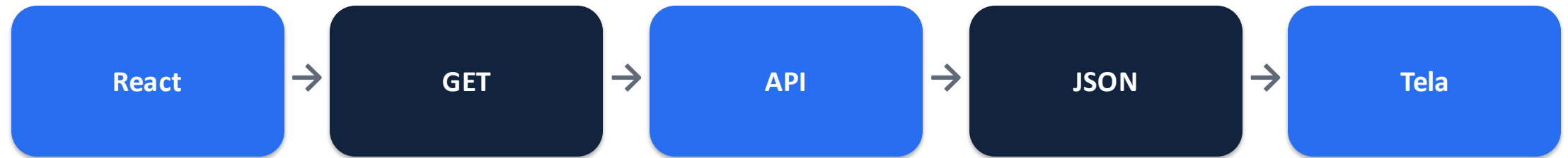


Agora entra a API

- Uma API permite que nossa aplicação obtenha dados de outro sistema
- Nossa aplicação fará uma requisição HTTP
- A API devolverá dados
- Normalmente recebemos esses dados em JSON

React conversando com uma API

Nosso objetivo é buscar produtos e transformá-los em elementos visuais.



Exemplo de resposta JSON

```
[  
  {  
    "id": 1,  
    "title": "Notebook",  
    "price": 3500  
  },  
  {  
    "id": 2,  
    "title": "Mouse",  
    "price": 100  
  }  
]
```

O que é useEffect?

- useEffect permite executar uma ação em determinado momento do componente
- Na nossa aula, vamos usá-lo quando a tela for carregada
- Nesse momento faremos a consulta da API
- Não precisamos decorar tudo agora: o importante é entender o fluxo

Primeiro useEffect

O [] será usado para executar esse efeito quando o componente iniciar.

```
import { useEffect } from "react";

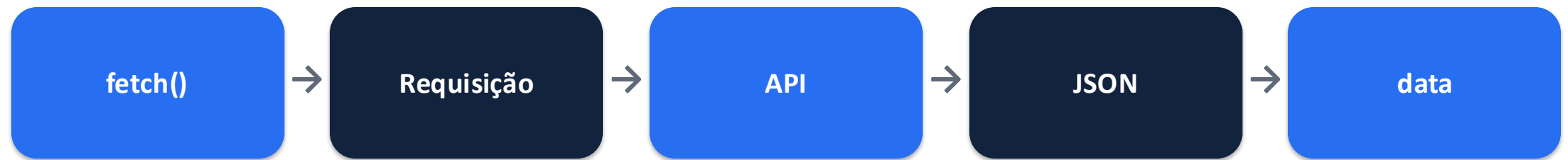
useEffect(() => {
  console.log("A aplicação iniciou");
}, []);
```

Consultando uma API

Primeiro vamos apenas observar os dados chegando no Console do navegador.

```
useEffect(() => {  
  fetch("https://fakestoreapi.com/products")  
    .then(response => response.json())  
    .then(data => console.log(data));  
}, []);
```

O que o fetch está fazendo?



Próximo passo: guardar os produtos

- Receber os dados não é suficiente
- Precisamos guardar a lista dentro do componente
- Para isso vamos usar useState
- Quando guardarmos os produtos, o React poderá atualizar a tela

Criando o estado de produtos

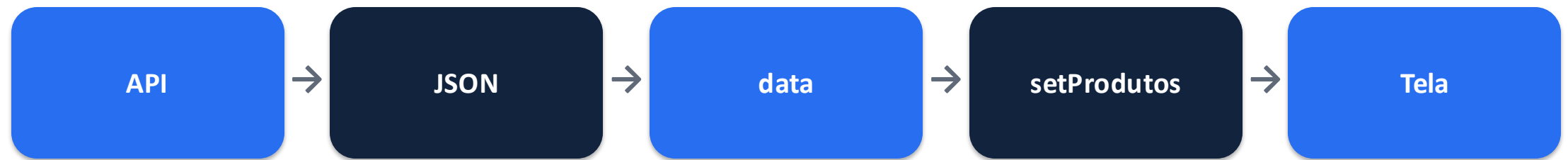
Começamos com uma lista vazia porque a API ainda não respondeu.

```
const [produtos, setProdutos] = useState([]);
```

API + estado

```
useEffect(() => {  
  fetch("https://fakestoreapi.com/products")  
    .then(response => response.json())  
    .then(data => {  
      setProdutos(data);  
    });  
}, []);
```

Fluxo completo da consulta



Como mostrar uma lista?

- A API devolve vários produtos
- Precisamos transformar cada produto em um elemento da interface
- Para isso usamos `map()`
- Cada item recebido pode gerar um bloco na tela

Mostrando os produtos

```
return (  
  <div>  
    <h1>Minha Loja</h1>  
  
    {produtos.map(produto => (  
      <div key={produto.id}>  
        <h2>{produto.title}</h2>  
        <p>$ {produto.price}</p>  
      </div>  
    ))}  
  </div>  
);
```

Do JSON para a interface



App.jsx completo da demonstração

Aqui juntamos os principais conceitos da aula.

```
import { useEffect, useState } from "react";

function App() {
  const [produtos, setProdutos] = useState([]);

  useEffect(() => {
    fetch("https://fakestoreapi.com/products")
      .then(response => response.json())
      .then(data => setProdutos(data));
  }, []);

  return (
    <div>
      <h1>Minha Loja</h1>

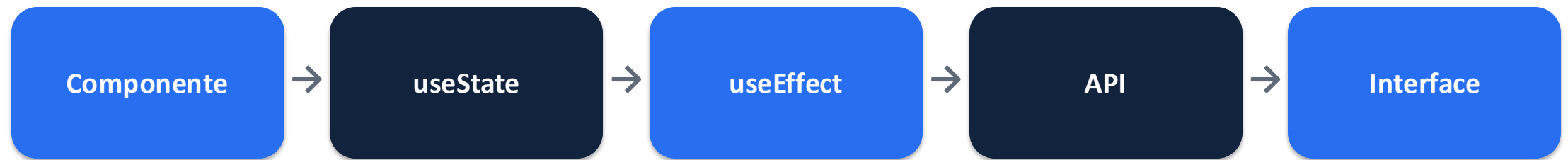
      {produtos.map(produto => (
        <div key={produto.id}>
          <h2>{produto.title}</h2>
          <p>$ {produto.price}</p>
        </div>
      )}
```

O que aprendemos nesse projeto?

- Criamos uma aplicação React com Vite
- Criamos componentes
- Entendemos que estado representa dados que podem mudar
- Usamos useEffect para iniciar uma consulta
- Usamos fetch para chamar uma API
- Guardamos a resposta no estado e mostramos os produtos na tela

Resumo da aula

Esse é o fluxo que o aluno precisa sair entendendo.



Próximos passos

- Melhorar o visual com CSS
- Criar um componente Produto reutilizável
- Adicionar uma mensagem de carregamento
- Tratar erros da API
- Depois avançar para formulários, rotas e outros hooks

Fim da aula

React fica mais simples quando entendemos o fluxo: Dados → Estado → Componentes → Interface

Prof. Lucas Lima

Laboratório: o que vamos construir

Agora vamos fazer tudo passo a passo, incluindo pastas e arquivos.

- Uma aplicação chamada minha-loja
- O React vai consultar uma API de produtos
- A resposta será guardada no estado da aplicação
- Cada produto será enviado para um componente Produto
- No final veremos nome e preço dos produtos no navegador

Passo 1 — Criar o projeto

Depois do `npm run dev`, abra o endereço mostrado pelo terminal.

```
npm create vite@latest minha-loja
```

```
Escolha:
```

```
Framework: React
```

```
Variant: JavaScript
```

```
cd minha-loja
```

```
npm install
```

```
npm run dev
```

Passo 2 — Estrutura que vamos usar

Criaremos apenas duas pastas: components e services.

```
minha-loja/
├── src/
│   ├── components/
│   │   └── Produto.jsx
│   ├── services/
│   │   └── produtoService.js
│   ├── App.jsx
│   ├── main.jsx
│   └── index.css
├── package.json
└── index.html
```

Para que serve cada arquivo?

- App.jsx → tela principal e estado da lista de produtos
- components/Produto.jsx → desenha um único produto
- services/produtoService.js → faz a chamada HTTP para a API
- main.jsx → ponto de entrada que inicia o React
- index.css → estilos; não é importante para entender a API

Passo 3 — Criar as pastas

No VS Code você também pode clicar com o botão direito em src → New Folder.

Dentro da pasta minha-loja

```
mkdir src/components
```

```
mkdir src/services
```

Por que criar uma pasta services?

- A chamada da API não fica misturada com o código da tela
- App.jsx fica mais fácil de ler
- Se o endereço da API mudar, sabemos onde procurar
- Para iniciantes, pense em services como: 'arquivos que conversam com APIs'

Passo 4 — Criar produtoService.js

Esse arquivo tem uma responsabilidade: buscar os produtos.

```
// src/services/produtoService.js

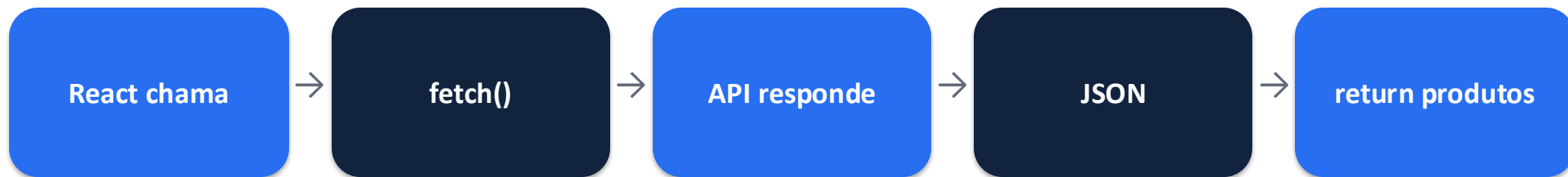
export async function buscarProdutos() {

  const response = await fetch(
    "https://fakestoreapi.com/products"
  );

  const produtos = await response.json();

  return produtos;
}
```

O que buscarProdutos() faz?



Passo 5 — Criar Produto.jsx

Esse componente apenas recebe dados e mostra um produto.

```
// src/components/Produto.jsx

function Produto({ nome, preco }) {
  return (
    <div>
      <h2>{nome}</h2>
      <p>$ {preco}</p>
    </div>
  );
}

export default Produto;
```

Entendendo Produto.jsx

- Produto é um componente React
- nome e preco chegam através das props
- O componente não chama a API
- Ele apenas recebe um produto e mostra suas informações

Passo 6 — Preparar App.jsx

Primeiro importamos tudo que o App vai precisar.

```
// src/App.jsx

import { useEffect, useState } from "react";
import Produto from "../components/Produto";
import { buscarProdutos } from "../services/produtoService";

function App() {

  const [produtos, setProdutos] = useState([]);

  return (
    <div>
      <h1>Minha Loja</h1>
    </div>
  );
}

export default App;
```

O que significa `useState([])`?

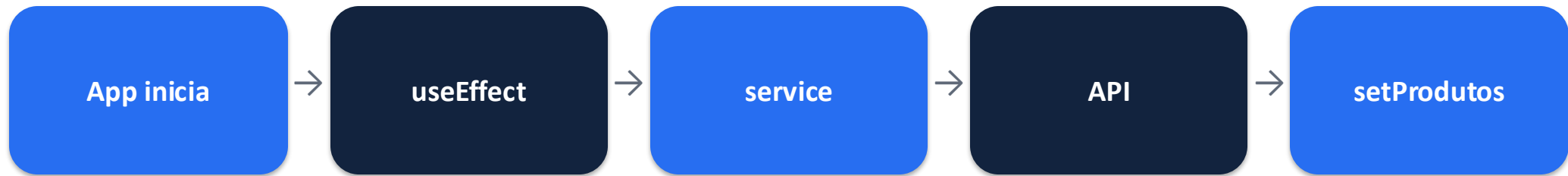
- `produtos` → variável onde ficará a lista
- `setProdutos` → função usada para atualizar a lista
- `[]` → começa vazio porque ainda não consultamos a API
- Quando `setProdutos` atualizar a lista, React renderiza a tela novamente

Passo 7 — Chamar a API no App.jsx

Coloque esse useEffect dentro da função App, antes do return.

```
useEffect(() => {  
  async function carregarProdutos() {  
    const dados = await buscarProdutos();  
    setProdutos(dados);  
  }  
  carregarProdutos();  
}, []);
```

Agora o fluxo está ficando completo



Passo 8 — Mostrar a lista no return

map percorre a lista e cria um componente Produto para cada item.

```
return (  
  <div>  
    <h1>Minha Loja</h1>  
  
    {produtos.map(produto => (  
      <Produto  
        key={produto.id}  
        nome={produto.title}  
        preco={produto.price}  
      />  
    ))}  
  
  </div>  
);
```

Passo 9 — App.jsx completo

Esse é o código principal que os alunos devem entender.

```
import { useEffect, useState } from "react";
import Produto from "../components/Produto";
import { buscarProdutos } from "../services/produtoService";

function App() {
  const [produtos, setProdutos] = useState([]);

  useEffect(() => {
    async function carregarProdutos() {
      const dados = await buscarProdutos();
      setProdutos(dados);
    }

    carregarProdutos();
  }, []);

  return (
    <div>
      <h1>Minha Loja</h1>

      {produtos.map(produto => (
        <Produto
          key={produto.id}
          nome={produto.title}
          preco={produto.price}
        />
      ))}
    </div>
  );
}
```

Passo 10 — Conferir main.jsx

O Vite já cria este arquivo. Aqui apenas conferimos que App está sendo iniciado.

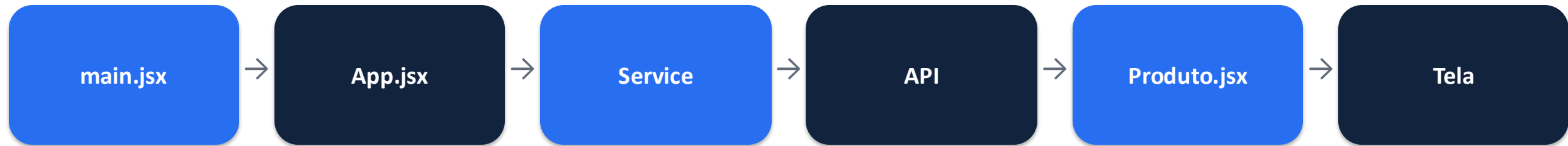
```
// src/main.jsx

import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Fluxo final da aplicação

main inicia App → App chama service → API devolve dados → Produto mostra cada item.



Onde cada responsabilidade fica?

- `main.jsx` → inicia a aplicação
- `App.jsx` → controla a tela e guarda a lista
- `produtoService.js` → conversa com a API
- `Produto.jsx` → apresenta um produto
- Essa separação ajuda o aluno a saber exatamente onde procurar cada coisa

Como testar durante a aula

- Execute `npm run dev`
- Abra o navegador no endereço informado pelo Vite
- Pressione F12 e abra Console para acompanhar erros
- Primeiro teste `console.log(dados)` depois da chamada da API
- Depois use `setProdutos(dados)` e renderize com `map`

Teste intermediário importante

Antes de montar a tela, confirme no Console que a API realmente devolveu os produtos.

```
async function carregarProdutos() {  
    const dados = await buscarProdutos();  
    console.log(dados);  
    setProdutos(dados);  
}
```

Se os produtos não aparecerem

- Veja se npm run dev está executando
- Confira o Console do navegador com F12
- Confira se produtoService.js está dentro de src/services
- Confira os imports e os nomes dos arquivos
- Abra a URL da API no navegador para verificar se ela responde

Ordem recomendada para ensinar

- 1. Criar o projeto com Vite
- 2. Limpar App.jsx e mostrar uma mensagem simples
- 3. Criar Produto.jsx com dados fixos
- 4. Criar produtoService.js e testar a API no console
- 5. Criar o estado produtos
- 6. Usar useEffect para carregar os dados
- 7. Usar map para mostrar os produtos

O aluno precisa guardar este raciocínio

